

我们先不讨论方程的解存在性问题，先假设解是存在的，主要考察数值求解的算法。其数值求解方法有1.迭代法 2.共轭梯度法(CG) 3.直接分解法(LLT)，下面我们将进行一一介绍

1.迭代法

这是最基础线性方程组数值求解方法，我们先将方程 $Ax = b$ 展开

$$\begin{aligned}a_{1,1}x_1 + a_{1,2}x_2 + \cdots + a_{1,n}x_n &= b_1, \\a_{2,1}x_1 + a_{2,2}x_2 + \cdots + a_{2,n}x_n &= b_2, \\&\vdots \\a_{n,1}x_1 + a_{n,2}x_2 + \cdots + a_{n,n}x_n &= b_n.\end{aligned}$$

根据不动点迭代的思路，可以进行如下变形从而有以下的**Jacobi 方法**

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j \neq i} a_{ij} x_j^{(k)} \right)$$

对上面的方法进行并行化后，可以得到更加高效的**Gauss-Seidel方法**

$$x_i^{(k+1)} = \frac{1}{a_{ii}} \left(b_i - \sum_{j=1}^{i-1} a_{ij} x_j^{(k+1)} - \sum_{j=i+1}^n a_{ij} x_j^{(k)} \right)$$

可以证明如果矩阵 A 是对角占优的则迭代将收敛，上面迭代法适用范围广算法也简单，但是实际使用时效率是偏低的。

2.共轭梯度法（Conjugate gradients）

其基本思路是将线性方程组 $Ax = b$ 的求解转化为如下二次型的最优化问题

$$\Phi(x) = \frac{1}{2} x^T A x - b^T x$$

梯度下降路径的选择是关键

$$\begin{aligned}k &= 0, \quad r^{(0)} = p^{(0)} = Ax^{(0)} - b, \\&\text{while } \left(\|Ax^{(k)} - b\| \leq \epsilon \right) \text{ and } (k < k_{\max}), \\&\quad \alpha = \frac{r^{(k)T} r^{(k)}}{p^{(k)T} A p^{(k)}}, \\&\quad x^{(k+1)} = x^{(k)} + \alpha p^{(k)}, \\&\quad r^{(k+1)} = r^{(k)} - \alpha A p^{(k)}, \\&\quad \beta = \frac{r^{(k+1)T} r^{(k+1)}}{r^{(k)T} r^{(k)}}, \\&\quad p^{(k+1)} = r^{(k+1)} + \beta p^{(k)}, \\&\quad k \leftarrow k + 1.\end{aligned}$$

关键不是“盲目沿负梯度前进”，而是在逐步扩展的 **Krylov 子空间里寻找最优方向**，从而避免普通梯度下降常见的锯齿路径。

实现时通常配合如下停止准则：

- 相对残差 $\|r^{(k)}\|/\|b\| < \epsilon$
- 达到最大迭代次数 $k_{\max}$

- 连续若干步残差下降幅度很小（提前停止）

3.稀疏对称矩阵的 sparse Cholesky 分解 $A = LL^T$

$$Ax = b \Leftrightarrow LL^T x = b \Leftrightarrow \begin{cases} Ly = b, \\ L^T x = y. \end{cases}$$

$$\begin{bmatrix} A_{II} & A_{IB} \\ A_{BI} & A_{BB} \end{bmatrix} = \begin{bmatrix} L_{II} & 0 \\ L_{BI} & L_{BB} \end{bmatrix} \begin{bmatrix} L_{II}^T & L_{BI}^T \\ 0 & L_{BB}^T \end{bmatrix}$$

需要注意的是Cholesky分解和CG迭代都要求矩阵 A 是对称正定的(Positive Definite),其他分解包括 LU 分解等,但效率不如 Cholesky 分解

$$Ax = b \Leftrightarrow LUx = b \Leftrightarrow \begin{cases} Ly = b, \\ Ux = y. \end{cases}$$

实际工程经验

- 对称正定优先 Cholesky（更快、更省内存）
- 非对称或不定矩阵转用 LU / QR
- 超大规模问题优先考虑“预条件 + 迭代法”

Poisson方程

Poisson 方程在自然界中应用广泛，比如热的扩散。其定义为 $\Delta a = b$ ，Laplace 方程是其中 $b = 0$ 的特殊情况，是一种椭圆型偏微分方程，用算法进行数值计算时，将其离散化后可以通过**稀疏线性系统**来进行求解。

Poisson 方程限定了边界后，其边界条件有 **Dirichlet边界条件**与 **Neumann边界条件**。

Dirichlet 边界条件

$$\begin{aligned} \Delta a &= \phi \quad \text{on } M, \\ a &= g \quad \text{on } \partial M. \end{aligned}$$

Neumann 边界条件

$$\begin{aligned} \Delta a &= \phi \quad \text{on } M, \\ \frac{\partial a}{\partial n} &= h \quad \text{on } \partial M. \end{aligned}$$

在三角网格上进行离散化后即转化为如下线性方程组

$$Aa = P\phi.$$

其中 $A \in \mathbb{R}^{V \times V}$ 为 **cotan-Laplace 矩阵**。非对角元与对角元常用 cotangent 权表示为

$$\begin{aligned} A_{ij} &= -\frac{1}{2}(\cot \beta_p^{ij} + \cot \beta_q^{ij}), \\ A_{ii} &= -\sum_{ij \in E} A_{ij}. \end{aligned}$$

P 是 Mass 矩阵，在展平的场景下可设 $P = E$ 。

将网格顶点分为内部点与边界点，可对矩阵 A 分块。

对于**Neumann 边界条件**，Poisson 问题可写为分块形式

$$\begin{bmatrix} A_{II} & A_{IB} \\ A_{IB}^T & A_{BB} \end{bmatrix} \begin{bmatrix} a_I \\ a_B \end{bmatrix} = \begin{bmatrix} \phi_I \\ \phi_B - h \end{bmatrix}.$$

对于**Dirichlet 边界条件**，有 $a_B = g \in \mathbb{R}^B$ ，消去边界未知量后得到仅关于内部变量的方程

$$A_{II} a_I = \phi_I - A_{IB} g.$$

Appendix

1.解的存在性

对于 $b = 0$ 为齐次线性方程组,线性方程组解的存在性等价于其矩阵 A 的可逆性，对于 $b \neq 0$ 非齐次线性方程组解的存在唯一性，由非线性方程组相容性问题 $r(A|b) = r(A)$ 的情况来判定。

2.高性能的C++线性代数库—Eigen库

Eigen自带了如下一些求解器，适用于求解大规模稀疏系统

求解器	适用矩阵	依赖	特点
SimplicialLDLT	对称正定	内置	轻量，适合 2D 问题
SimplicialLLT	对称正定	内置	比 LDLT 更快但更不稳定
SparseLU	任意方阵	内置	基于 SuperLU，支持非对称
SparseQR	任意矩阵	内置	用于最小二乘，内存高

```
#include <Eigen/Sparse>
#include <Eigen/SparseLU>

Eigen::SparseMatrix<double> A(n, n);
// ... 矩阵装配 A ...

Eigen::VectorXd b(n);
// ... 矩阵装配 b ...

Eigen::SparseLU<Eigen::SparseMatrix<double>> solver;
solver.compute(A);

if (solver.info() != Eigen::Success) {
    std::cerr << "Decomposition failed!" << std::endl;
    return -1;
}

Eigen::VectorXd x = solver.solve(b);
```

参考：

经典教材《Numerical Optimization》(Nocedal & Wright)